

研讨题16-2

进程管理

程序vs进程

- 一个程序可以对应多个进程吗？–可以

应用 (11)							
>	Adobe Acrobat DC (32 位) (4)		0%	161.9 MB	0 MB/秒	0 Mbps	0%
>	Clash Verge		0%	14.2 MB	0 MB/秒	0 Mbps	0%
>	Google Chrome (44)	🗑	0%	1,614.5 ...	0 MB/秒	0 Mbps	0% GPU 0 - 3D
>	Google NotebookLM (6)	🗑	0%	59.5 MB	0.1 MB/秒	0 Mbps	0%
>	Joplin (8)	🗑	0%	213.9 MB	0.1 MB/秒	0 Mbps	0.1% GPU 0 - 3D
>	Microsoft Edge (40)	🗑	0%	804.4 MB	0 MB/秒	0 Mbps	0.1% GPU 0 - 3D
>	Notion (8)	🗑	0%	468.1 MB	0 MB/秒	0 Mbps	0%
>	Visual Studio Code (112)	🗑	0%	340.2 MB	0 MB/秒	0 Mbps	0%
>	Windows Terminal Host		0%	15.0 MB	0 MB/秒	0 Mbps	0.4% GPU 0 - 3D
>	Windows 资源管理器		0%	97.3 MB	0.1 MB/秒	0 Mbps	0.1% GPU 0 - 3D
>	任务管理器		0.5%	120.8 MB	0.1 MB/秒	0 Mbps	0.1% GPU 0 - 3D

- 一个进程可以对应多个程序吗？–不能

同一时间点上，一个进程只能与一个程序相关联

但进程可以通过调用 exec 函数来加载一个新的程序， 如在bash中输入ls，执行的程序会从/bin/bash变为/bin/ls

进程切换

进程上下文

a) 用户级上下文

- 程序代码（Code Segment）
- 用户栈（User Stack）
- 全局变量 / 数据段（Data Segment）
- 堆（Heap）
- 打开的文件描述符表

b) 寄存器上下文

- 程序计数器（PC / RIP） → 指示下一条要执行的指令地址；
- 栈指针（SP / RSP） → 指示当前栈顶；
- 通用寄存器
- 状态寄存器（标志位）；

c)内核级上下文

- 内核栈 (Kernel Stack)
- 内核栈指针
- 内核态寄存器 (系统调用或中断时使用)
- 内核数据结构 (PCB、调度信息、信号、内存管理信息、文件表指针等)

切换过程

将 P_old 的所有硬件上下文 (PC, 寄存器等) 保存到 P_old 在内核中的进程控制块 (PCB) 里, 之后加载新进程的上下文

必要性

实现多任务并行

多核的上下文切换

- 需要考虑进程调度问题, 如哪个核空闲, 切换进程到另一个核是否会带来缓存一致性损失

fork()函数

example

```
int main(int argc, char** argv)
{
    pid_t pid;
    int x = 1;

    pid = Fork();
    if (pid == 0) {    /* Child */
        printf("child : x=%d\n", ++x);
        printf("child : x=%d\n", ++x);
        return 0;
    }

    /* Parent */
    printf("parent: x=%d\n", --x);
    printf("parent: x=%d\n", --x);
    return 0;
}
```

```
linux> ./forkx2
parent: x=0
parent: x=-1
child : x=2
child : x=3
```

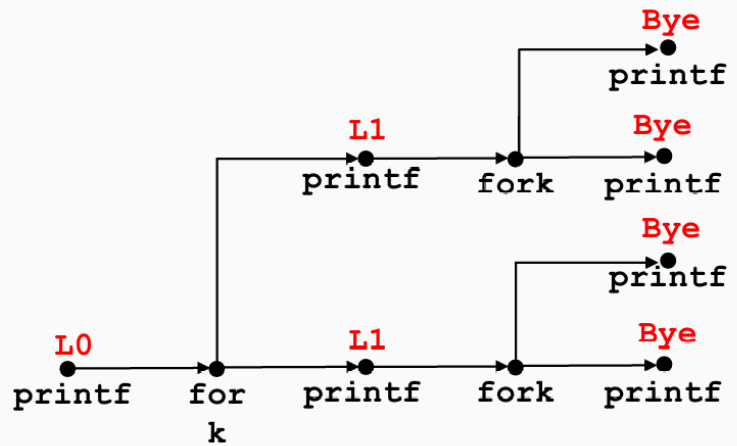
parent -> child (0 -1 2 3)

child -> parent (2 3 2 1)

```

void fork2()
{
    printf("L0\n");
    fork();
    printf("L1\n");
    fork();
    printf("Bye\n");
}
forks.c

```



output: L0 L1 BYE BYE L1 BYE BYE

程序终止和回收

终止原因

- 自愿终止: return exit()
- 非自愿终止: 接收到致命信号或由内核终止

终止步骤

- 刷新和关闭 I/O 缓冲区
- 释放大部分内存: 进程的用户空间内存（代码、数据、堆、栈）会被标记为可用，并归还给系统
- 保留一个PCB来保存该进程的PID和退出状态码 (exit status)
- 通知父进程: 内核向该进程的父进程发送一个 SIGCHLD 信号

wait()/waitpid()

等待直到wait() 捕获到一个僵尸子进程

之后内核将子进程退出状态码从其 PCB 中复制给调用 wait() 的父进程彻底释放僵尸子进程的 PCB

若没有回收则僵尸子进程会一直存在，大量僵尸进程可能导致内核资源泄漏或系统崩溃

Another wait Example

- If multiple children completed, will take in arbitrary order
- Can use macros WIFEXITED and WEXITSTATUS to get information about exit status

```
void fork10() {
    pid_t pid[N];
    int i, child_status;

    for (i = 0; i < N; i++)
        if ((pid[i] = fork()) == 0) {
            exit(100+i); /* Child */
        }
    for (i = 0; i < N; i++) { /* Parent */
        pid_t wpid = wait(&child_status);
        if (WIFEXITED(child_status))
            printf("Child %d terminated with exit status %d\n",
                wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminate abnormally\n", wpid);
    }
}
```

forks.c

循环N次，乱序回收

execve(char *filename, char *argv[], char *envp[])

- filename 执行的目标文件名
- argv 参数列表
- envp 环境变量 key = value 的键值对

调用execve后程序会执行新的主函数

```
int main(int argc, char **argv, char **envp);
```

因此可以通过fork() + execve()用创建进程的方式运行一个指定的程序文件